

Embedding and Embedding Finetuning

Chapter-1

1. What is an Embedding?
2. What is Semantic Search(Similarity Search)?
3. Keyword vs Semantic Search(Similarity Search)
4. Encoding vs Embedding
5. Application of Embedding

fundamental

Chapter-2

1. All model for embedding
2. Types of Embedding Models
3. How to select best embedding model -
4. Embedding model leader board

model analysis

Chapter-3

1. Is embedding model transformer based model
2. On which dataset those embedding models trained?

architecture

Chapter-4

1. What is embedding model finetuning
2. Why & when we should do the embedding finetuning?

Chapter-5

1. Data set format for the embedding finetuning?
2. Finetune any embedding model on your on custom data

Chapter-6

1. Embedding Fine-Tuning vs LLM Fine-Tuning

What is an Embedding?

Embedding is mathematical representation of the data

Contextual

Embedding is a way to convert text, image, audio, video into meaningful numbers.

Embedding is dense, high-dimensional numerical vector that represents the semantic meaning of data.

Dense: Non Spars → non zero

High-dimensional: 384, 768, 1024, 1536, 2048 etc....

vector: Mathematical object in linear algebra In programming this vector is typically stored as an array, means a collection of numerical values(float).

The capital of France is Paris	→	0.10	-34	-52	-77	-0.40	-0.28	-0.19
Berlin is Germany's capital	→	0.00	-35	-52	-0.65	-0.91	-0.03	-0.29

1x7
Column
dim
features

my name is sunny →



Contextual
Semantic

Semantic Search

Similarity Search

What is Semantic Search?

Semantic search = Meaning-based intelligent search using embeddings.

Semantic search means searching based on meaning, not just exact words

Semantic search is usually performed using:

Cosine similarity
Dot product
Euclidean

embedding

{
Sentence 1 (doc)
Sentence 2
}

Dimension

Features

Example:

- eating fiber reduces heart risk ← Sent 1
- eating fruits and vegetables lowers cardiovascular disease chances ← Sent 2

Both sentences have the same meaning

Even though the words are different, embeddings place them close together in vector space.

That's why semantic search returns relevant results even when wording changes.

Example:

- Eating fiber reduces heart risk = Sent 1
- Buying a new car improves driving comfort = Sent 2

Semantic

Completely different topics

Health vs Automobile

Embeddings will be far apart

Low cosine similarity

Keyword vs Semantic Search (Similarity Search)

Clean Comparison Line

Keyword search matches words.

Semantic search matches meaning: find out similarity using matrix like cosine, Euclidian, dot product etc.

How Keyword-Based Search Work

If you search:

"How to reduce heart risk"

The system will mainly look for sentence containing those exact words like:

- reduce
- heart
- risk

If a sentence instead contains:

"Ways to lower cardiovascular disease chances"

A traditional keyword search may not match it because:

- reduce $\neq$ lower
- heart risk $\neq$ cardiovascular disease

Even though the meaning is similar, the words are different

Example

Sen1: how to lose weight fast

Sen2: quick fat loss strategies

Keyword search $\rightarrow$ weak match

Semantic search $\rightarrow$ strong match

Encoding vs Embedding

Encoding: Converts data into a numerical format so that machines can process it.

Count based, matrix factorization based

One-hot encoding, BOW, TF-IDF, BM25, GloVe

One-hot-encoding

id	color	id	color_red	color_blue	color_green
1	red	1	1	0	0
2	blue	2	0	1	0
3	green	3	0	0	1
4	blue	4	0	1	0

Parse

Zero

Document D1	The child makes the dog happy
	the: 2, dog: 1, makes: 1, child: 1, happy: 1
Document D2	The dog makes the child happy
	the: 2, child: 1, makes: 1, dog: 1, happy: 1

	child	dog	happy	makes	the	BoW Vector representations
D1	1	1	1	1	2	[1,1,1,1,2]
D2	1	1	1	1	2	[1,1,1,1,2]

Vocab

unique

count

1 Term Frequency (TF)

$$TF(\text{term}, \text{document}) = \frac{\text{Number of times the term appears in the document}}{\text{Total number of terms in the document}}$$

NLP

2 Inverse Document Frequency (IDF)

$$IDF(\text{term}) = \log \left(\frac{\text{Total number of documents in the corpus}}{1 + \text{Number of documents containing the term}} \right)$$

3 TF-IDF (Final Formula)

$$TF-IDF(\text{term}, \text{document}) = TF(\text{term}, \text{document}) \times IDF(\text{term})$$

number

Embedding: Converts data into a numerical vector that preserves meaning .

Embedding("king") Embedding("queen") Distance between them → small Angle will be less between two vector

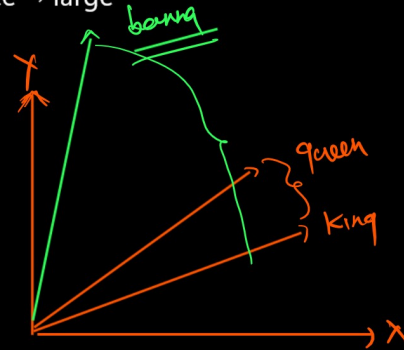
Embedding("king") Embedding("banana") angle between huge Distance → large

Classical Word Embedding Models

- Word2Vec, , FastText

Limitation:

- They create embeddings at the word level only.
- They produce static embeddings (fixed vectors).
- They do not understand context.



<u>man</u>	→	0.6	-0.2	0.8	0.9	-0.1	-0.9	-0.7
woman	→	0.7	0.3	0.9	-0.7	0.1	-0.5	-0.4
king	→	0.5	-0.4	0.7	0.8	0.9	-0.7	-0.6
queen	→	0.8	-0.1	0.8	-0.9	0.8	-0.5	-0.9

Word Word embedding

Henry is a AI eng.



SOTA Transformer-based Embeddings Model

word2vec →

NN

- Based on Transformer architecture
- Contextual embeddings Context-aware embeddings.
- Any text (word, phrase, or sentence, Full paragraphs)

Example: Sentence Transformers Models, MiniLM, BGE, OpenAI Embeddings, Gemini Embedding, Claude Embedding etc....

The capital of France is Paris →	0.10	-34	-52	-77	-0.40	-0.28	-0.19
Berlin is Germany's capital →	0.00	-35	-52	-0.65	-0.91	-0.03	-0.29

Summary →
Summary is a AI eng →
Paragraph
text

Example

Sentence 1: I sat on the river bank.

Sentence 2: I deposited money in the bank.

Word: bank

meaning

word2vec

In a static embedding model (word2vec), the word "bank" is represented by the same vector in both sentences.
bank → [0.21, -0.44, 0.89, ...]

In SOTA Transformer-based Embeddings Model, the word "bank" will be represented by the different vector.

bank → [0.21, -0.44, 0.89, ...]

bank → [0.93, 0.01, 0.75, ...]

Embedding Models

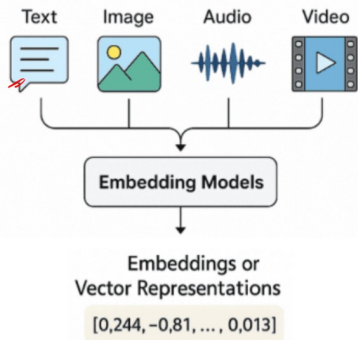
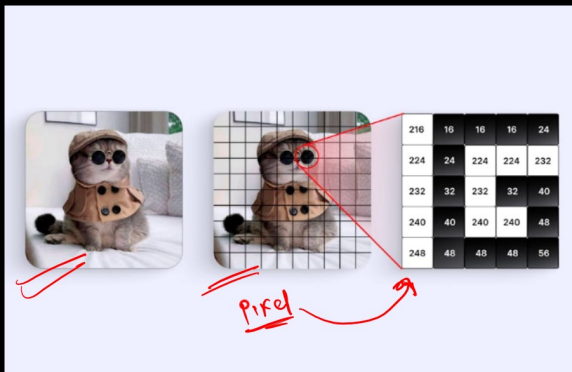


Image Embeddings: We can also convert images into embeddings using Vision or Multimodal models (CLIP, Vision Transformer(ViT))



Approach 1: Audio → Text → Text Embedding

Audio → ASR (Speech-to-Text) → Text → Text Embedding Model → Vector DB

OpenAI Whisper

Google Speech-to-Text

Meta MMS

Approach 2: Direct Audio Embedding (Advanced)

Audio → Audio Encoder Model → Audio Embedding → Vector DB



Google YAMNet

OpenAI CLAP

Meta wav2vec

Audio Embeddings

Use Cases

- Voice search
- Music similarity

Video Embeddings

Video = sequence of images + audio + time dimension

Video

- └─> Audio → ASR → Text → Text Embedding
- └─> Frames → Vision Encoder → Image Embedding
- └─> Metadata → Text Embedding

→ Multi-vector Retrieval

Direct Video Encoder

Video → Video Transformer Encoder → Video Embedding → Vector DB

Video Embedding Models

VideoMAE

TimeSformer

I3D

Use Cases

Video search

Content moderation

Recommendation (YouTube / Netflix)

Application of embedding

1. Semantic Search (Most Fundamental): Meaning-based search
Instead of matching keywords, we match vector similarity.

2. Google-like search engines

3. Google Image search

4. Pinterest

User Search: "a red car" System retrieves: "Images of red cars"

5. Recommendation Systems

Netflix: Similar movies

Amazon: Similar products

Spotify: Similar songs

All use vector similarity.

6. Topic Modeling (Clustering)

7. Group similar documents together

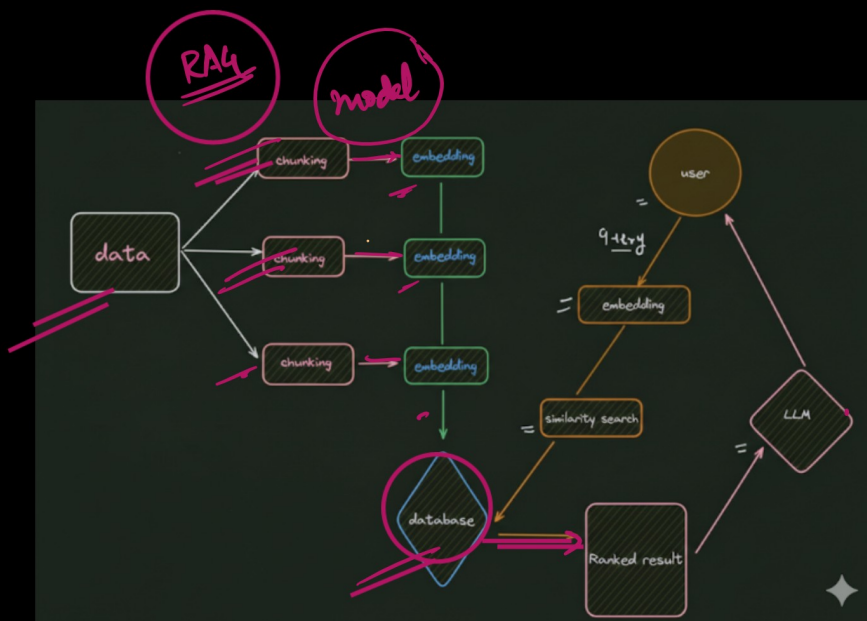
News categorization

8. RAG (Retrieval-Augmented Generation)

Semantic Search

Application

Doc 1 ↔ Doc 2
↕
Doc 3



Models for Embedding

Model Name	Modality	Provider	Category
text-embedding-3-small	Text	OpenAI	Closed
text-embedding-3-large	Text	OpenAI	Closed
text-embedding-ada-002 (legacy)	Text	OpenAI	Closed
Gemini Embeddings	Text	Google	Closed
Claude Embeddings	Text	Anthropic	Closed
Cohere embeddings	Text	Cohere	Closed
Titan Text Embeddings	Text	AWS (Amazon Bedrock)	Closed

all-mpnet-base-v2	Text	HuggingFace (Sentence-Transformers)	Open
all-distilroberta-v1	Text	HuggingFace (Sentence-Transformers)	Open
all-roberta-large-v1	Text	HuggingFace (Sentence-Transformers)	Open
msmarco-minilm-v3	Text	HuggingFace (Sentence-Transformers)	Open
BAAI-BGE-Base-En	Text	HuggingFace (BAAI)	Open
BAAI-BGE-Large	Text	HuggingFace (BAAI)	Open
BGE-multilingual	Text	HuggingFace (BAAI)	Open
e5-small	Text	HuggingFace	Open
e5-base	Text	HuggingFace	Open
e5-large	Text	HuggingFace	Open
st5-xd-embed	Text	HuggingFace	Open
miniLM L6	Text	Microsoft(OpenAI)	Open
miniLM L12	Text	Microsoft(OpenAI)	Open
OpenCLIP	Image (Vision)	HuggingFace (OpenAI)	Open
BLIP2 embedding models	Image (Vision)	Various	Open
DINO-v2 embeddings	Image (Vision)	Meta	Open
Wav2Vec 2.0	Audio	Meta	Open
HuBERT	Audio	Meta	Open
YAMNet	Audio	Google	Open
CLAP (Contrastive Language-Audio Pretraining)	Audio	LAION	Open
VideoMAE	Video	Meta	Open
I3D (Inflated 3D ConvNet)	Video	DeepMind	Open

How to select best embedding model

1. Embedding Quality (Most Important): Check benchmark performance:

Look at:

- MTEB leaderboard
- BEIR benchmark

} next video

2. Dimensionality

Example:

- 384 dim → lightweight
- 768 dim → balanced
- 1536 dim → high quality but heavy

→ good result

Higher dimension:

- Better semantic representation
- But more storage cost
- More memory usage
- Speed impact

} Saves

3. Cost

Close source (Open AI embedding):

- Easy, high quality, API cost

Open source:

- Free but infra + scaling cost

4. Domain Suitability

General model:

- all-MiniLM
- OpenAI embedding

Embedding model leader board

MTEB Leaderboard (Hugging Face Space)

This shows various embedding models ranked across different tasks.

MTEB Official Framework & Leaderboard Repo (GitHub)

MTEB Homepage & Info

Explanation of the benchmark and how it evaluates models.

HF Sentence Transformer Doc: <https://www.sbert.net/>

ST Repo: <https://huggingface.co/models?library=sentence-transformers&sort=downloads>

Pretrain Model → Massive Unlabeled Text → Pretraining Usually mixture of (Wikipedia, Common Crawl, Books Corpus, Web text, Open datasets, Public web pages)

↓

Contrastive Fine-Tuning (Contrastive Learning: Pairs of similar/dissimilar text are used)

Data set format for the embedding finetuning?

Case A — Pair Format

```
{  
  "sentence1": "text A",  
  "sentence2": "text B",  
  "label": 1    # 1 = similar, 0 = not similar  
}
```

Case B — Triplet Format (Anchor / Pos / Neg)

```
{  
  "query" or "anchor": "how to bake cake",  
  "positive_passage": "similar meaning text",  
  "negative_passage": "unrelated text"  
}
```

Case C — N-way format (Ranking)

```
{  
  "query": "question",  
  "positives": ["doc1", "doc2"],  
  "negatives": ["docA", "docB", "docC"]  
}
```

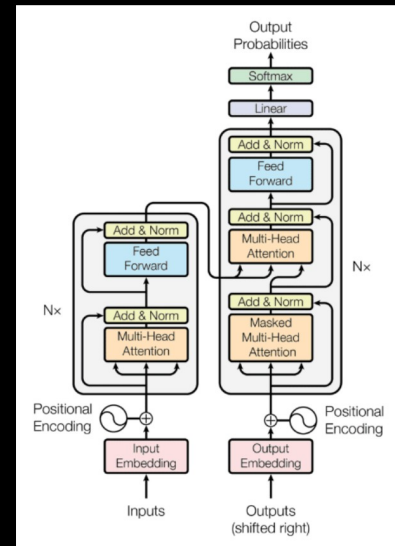
(Natural Questions, NLI datasets (SNLI, MNLI), STS Benchmark, BEIR benchmark datasets etc...)

Goal:

similar → close vectors
different → distant vectors

This is usually done with:

cosine similarity
triplet loss
contrastive loss (InfoNCE)



$$\text{sim}(u, v) = \frac{u \cdot v}{\|u\| \|v\|}$$

Triplet Loss

$$\mathcal{L} = \max(0, \text{sim}(q, d^-) - \text{sim}(q, d^+) + m)$$

$$\mathcal{L}_i = -\log \frac{e^{\text{sim}(q_i, d_i^+) / \tau}}{\sum_{j=1}^N e^{\text{sim}(q_i, d_j) / \tau}}$$

↓

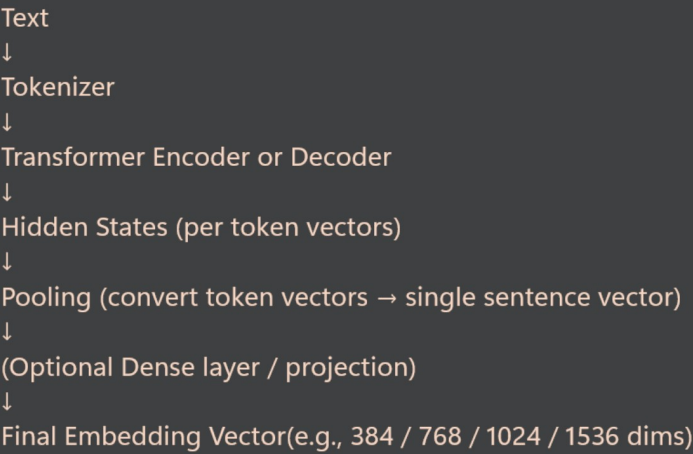
Final Embedding Model(our own custom tuning)

google-bert/bert-base-uncased

BAAI/bge-base-en-v1.5(Pretrained + Contrastive Fine-tuned embedding model)

<https://huggingface.co/datasets/sentence-transformers/all-nli>
<https://huggingface.co/datasets/sentence-transformers/stsb>
<https://huggingface.co/datasets/sentence-transformers/quora-duplicates>
<https://huggingface.co/datasets/sentence-transformers/natural-question>
<https://huggingface.co/datasets/shawhin/ai-job-embedding-finetuning>

Core Architectural Concept of an Embedding Model:



Architecture Category	Example
Encoder-Only Model	SBERT, E5, BGE
Cross Encoder	MS-MARCO Cross-Encoder
Multimodal Dual Encoder	CLIP
Decoder-Based Embedding	OpenAI Embeddings (text-embedding-3-large), Gemini Embeddings

Aspect	Dual Encoder	Cross Encoder
How it processes input	Query and document encoded separately → similarity computed (dot / cosine)	Query + document encoded together in one pass → direct relevance score
Speed & Scalability	Very fast, scalable for millions of docs	Slow, needs one forward pass per

What is embedding finetuning

Embedding fine-tuning means training an embedding model further on your specific data so that its vector representation becomes more accurate and relevant for your domain or task.

Embedding fine-tuning means training an embedding model so that its vector representations become more task-specific rather than generic.

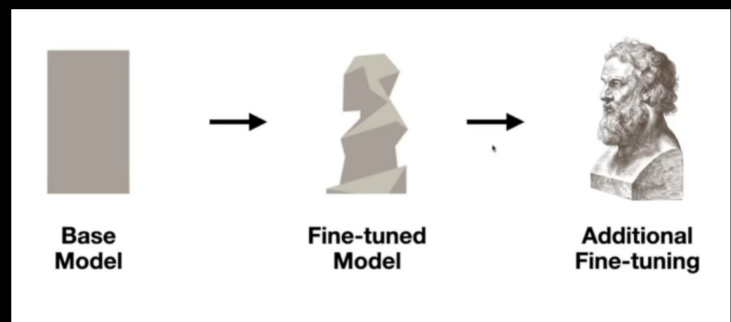
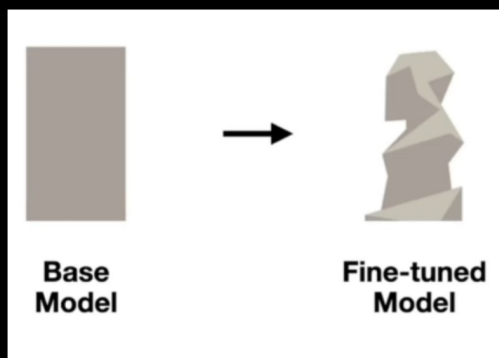
Normal Embeddings

- Pretrained on large generic data (Wikipedia, Common Crawl, QA pairs)
- Good for general search/semantic similarity

Fine-Tuned Embeddings

- Further trained on your specific dataset
- Model learns better semantic structure for your domain

So instead of using the original "out-of-the-box vectors," you customize them for your use-case.



Step 1: Gather positive (and negative) pairs

Step 2: Pick a pre-trained model

Step 3: Pick a loss function

Step 4: Fine-tune the model

Step 5: Evaluate the model

Query

"Data pipeline management, advanced SQL techniques, ETL/ELT processes."

Positive Match

Make your impact within a rapidly growing Fintech Company. Build and manage robust data pipelines that support scalable...

Negative Match

Collaborate to design and implement end-to-end AI and data science lifecycle, from data collection and preprocessing to model deployment and monitoring...

Here are the real reasons based on practical use-cases:

1. Domain-Specific Understanding(Medical, Finance etc...)

Generic embeddings treat all language alike.

Fine-tuned embeddings understand domain semantics better.

2. Better Search / Semantic Retrieval

If you are building:

- RAG (Retrieval Augmented Generation)

3. Task-Specific Nearness

- Intent detection
- Duplicate detection

Fine-tuning aligns vectors with task intent not just basic language semantics.

Aspect	Embedding Fine-Tuning	LLM Fine-Tuning
Core Purpose	Improve semantic vector space	Customize text generation
Training Focus	Similarity & ranking	Next token generation
Loss Function	Contrastive / Triplet / Ranking	Cross-Entropy
Dataset	Paired / Triplet	Chat / Instruction
Output	High-quality embeddings	High-quality responses
In RAG	Improves retrieval	Improves generation
Architecture	Encoder-only	Decoder-heavy / Causal

Thumbnail
(AI Copilot (vibe coding) to full Autonomy AI Agent-)

build AI Agent with vibe coding
End-to-end AI Project

- Sumit
- (1) GitHub (copilot) = Plugin → IDE VScode | Pycharm | IntelliJ PAID → Premium 10\$
 - (2) Cursor = IDE → PAID
 - (3) Autogpt →
 - (4) Replit →

lok

2hr



Asetic AI Code generate → Code Run → deploy
Customer Support

→ Rush 3 week 10 march
15 march